

RL_00_proyectos

April 10, 2026

1 Recorrido por los Proyectos de Reinforcement Learning

Este notebook relaciona cada proyecto, ejemplo y modelo del directorio con los conceptos teóricos.

1.1 Notebooks de Teoría (01_teoría/)

Notebook	Contenido	Nivel
RL_01_fundamentos.ipynb	Fundamentos, SARSA, Q-Learning	Básico
RL_02_dqn.ipynb	DQN, Gymnasium, Práctica	Intermedio
RL_03_sb3.ipynb	Stable-Baselines3, PPO, SAC	Avanzado

1.2 Mapa de Contenidos

NIVEL 1: TEORÍA (01_teoría/)

RL_01_fundamentos.ipynb
Conceptos Básicos (Agente, Entorno, Estado, Política)
SARSA (On-Policy)
Q-Learning (Off-Policy)

RL_02_dqn.ipynb
Deep Q-Network (DQN)
Gymnasium
Práctica Final

RL_03_sb3.ipynb
Stable-Baselines3
PPO, A2C, SAC, TD3
Entrenamiento Avanzado

NIVEL 2: FUNDAMENTOS (02_fundamentos/)

core/agentes.py QLearningAgent, SARSAAgent, DQNAgent

ejemplos/

ejemplo_qlearning_taxi.py	Q-Learning tabular
ejemplo_cartpole_dqn.py	DQN básico

NIVEL 3: PROYECTOS DQN (03_proyectos_dqn/)

nibbler/	Snake con DQN + Pygame
flappybird/	Timing crítico
racing/	Multi-agente, sensores

NIVEL 4: PROYECTOS AVANZADOS (04_proyectos_avanzados/)

lunarlander/	PPO, A2C, DQN (SB3)
highway/	Conducción autónoma
minigrid/	Observación parcial
pybullet/	Robótica 3D (SAC, TD3)

1.3 Estructura del Directorio

Reinforcement Learning/

01_teoría/	# NIVEL 1: Fundamentos teóricos
RL_01_fundamentos.ipynb	
RL_00_proyectos.ipynb	# Este notebook
02_fundamentos/	# NIVEL 2: Implementaciones base
core/	# Algoritmos
agentes.py	# Q-Learning, SARSA, DQN
utils.py	# Gráficos y evaluación
ejemplos/	# Tutoriales básicos
ejemplo_cartpole_dqn.py	
ejemplo_qlearning_taxi.py	
03_proyectos_dqn/	# NIVEL 3: Proyectos con DQN
nibbler/	# Snake con DQN
flappybird/	# Flappy Bird
racing/	# Carreras multi-agente
04_proyectos_avanzados/	# NIVEL 4: Algoritmos avanzados
lunarlander/	# PPO, A2C, DQN
highway/	# Conducción autónoma
minigrid/	# Laberintos 2D
pybullet/	# Robótica 3D
modelos/	# Pesos entrenados (.pth)

```
[1]: # Imports necesarios para este recorrido
import sys
import os
from pathlib import Path

# Añadir el directorio raíz al path (subimos un nivel desde 01_teoría/)
RL_DIR = Path().absolute().parent
if str(RL_DIR) not in sys.path:
    sys.path.insert(0, str(RL_DIR))

# También añadir 02_fundamentos para imports de core
FUND_DIR = RL_DIR / "02_fundamentos"
if str(FUND_DIR) not in sys.path:
    sys.path.insert(0, str(FUND_DIR))

print(f"Directorio RL: {RL_DIR}")
print(f"\nEstructura:")
for item in sorted(RL_DIR.iterdir()):
    if item.name.startswith('.'):
        continue
    prefix = " " if item.is_dir() else " "
    print(f" {prefix} {item.name}")
```

Directorio RL: /home/jeironpro/Documentos/OPT-ML2/Reinforcement Learning

Estructura:

```
01_teoría
02_fundamentos
03_proyectos_dqn
04_proyectos_avanzados
05_Practica
CLAUDE.md
modelos
requirements.txt
```

2 1. Módulo Core: Las Implementaciones Base

Ubicación: 02_fundamentos/core/

El módulo core/ contiene las implementaciones fundamentales de los algoritmos vistos en teoría.

2.1 Relación con la Teoría

Concepto Teórico	Implementación en core/agentes.py
Tabla Q	QLearningAgent.Q (defaultdict)
Ecuación de Bellman	QLearningAgent.aprender()

Concepto Teórico	Implementación en <code>core/agentes.py</code>
Política -greedy	<code>QLearningAgent.seleccionar_accion()</code>
Experience Replay	<code>ReplayBuffer</code> (deque circular)
Red Neuronal para Q	<code>DQNetwork</code> (<code>nn.Module</code>)
Target Network	<code>DQNAgent.target_network</code>
TD Error	Diferencia entre target y predicción

```
[2]: # Veamos las clases disponibles en core/agentes.py
from core import agentes
import inspect

print("="*60)
print("CLASES EN 02_fundamentos/core/agentes.py")
print("="*60)

clases = [
    ("ReplayBuffer", "Buffer circular para Experience Replay (DQN)"),
    ("QLearningAgent", "Agente Q-Learning tabular"),
    ("SARSAAgent", "Agente SARSA (on-policy)"),
    ("DQNetwork", "Red neuronal para aproximar Q(s,a)"),
    ("DQNAgent", "Agente DQN completo con target network"),
]

for nombre, desc in clases:
    if hasattr(agentes, nombre):
        print(f"\n {nombre}")
        print(f"   {desc}")
    else:
        print(f"\n {nombre} - No encontrado")
```

```
=====
CLASES EN 02_fundamentos/core/agentes.py
=====
```

```
ReplayBuffer
    Buffer circular para Experience Replay (DQN)
```

```
QLearningAgent
    Agente Q-Learning tabular
```

```
SARSAAgent
    Agente SARSA (on-policy)
```

```
DQNetwork
    Red neuronal para aproximar Q(s,a)
```

```
DQNAgent
```

Agente DQN completo con target network

```
[3]: # Ejemplo: Ver la implementación de Q-Learning
from core.agentes import QLearningAgent

print("="*60)
print("ANATOMÍA DE QLearningAgent")
print("="*60)

# Crear un agente de ejemplo
agente = QLearningAgent(n_estados=16, n_acciones=4, alpha=0.1, gamma=0.99,
    ↪epsilon=0.1)

print(f"""
Parámetros del agente:
    - n_estados: {agente.n_estados if hasattr(agente, 'n_estados') else 'N/A'}
    - n_acciones: {agente.n_acciones}
    - alpha (learning rate): {agente.alpha}
    - gamma (descuento): {agente.gamma}
    - epsilon (exploración): {agente.epsilon}

Correspondencia con la teoría:

     $Q(s,a) \leftarrow Q(s,a) + [r + \max_{a'} Q(s',a') - Q(s,a)]$ 

    alpha ( ) = {agente.alpha} → Tasa de aprendizaje
    gamma ( ) = {agente.gamma} → Factor de descuento
    epsilon ( ) = {agente.epsilon} → Probabilidad de exploración

""")
```

```
=====
ANATOMÍA DE QLearningAgent
=====
```

Parámetros del agente:

- n_estados: 16
- n_acciones: 4
- alpha (learning rate): 0.1
- gamma (descuento): 0.99
- epsilon (exploración): 0.1

Correspondencia con la teoría:

$$Q(s,a) \leftarrow Q(s,a) + [r + \max_{a'} Q(s',a') - Q(s,a)]$$

alpha () = 0.1 → Tasa de aprendizaje
gamma () = 0.99 → Factor de descuento

epsilon() = 0.1 → Probabilidad de exploración

```
[4]: # Ejemplo: Ver la estructura del ReplayBuffer (Experience Replay)
from core.agentes import ReplayBuffer

print("="*60)
print("EXPERIENCE REPLAY BUFFER")
print("="*60)

buffer = ReplayBuffer(capacidad=1000)

# Simular algunas experiencias
import numpy as np
for i in range(5):
    estado = np.array([i, i*2])
    accion = i % 2
    recompensa = 1.0 if i == 4 else -0.1
    siguiente = np.array([i+1, (i+1)*2])
    terminado = (i == 4)
    buffer.agregar(estado, accion, recompensa, siguiente, terminado)

print(f"""
Experiencias almacenadas: {len(buffer)}

¿Para qué sirve el Experience Replay?
1. Rompe la correlación temporal entre muestras consecutivas
2. Reutiliza experiencias pasadas (eficiencia de datos)
3. Estabiliza el entrenamiento de redes neuronales

Cada experiencia es una tupla (s, a, r, s', done):
""")

for i, exp in enumerate(list(buffer.buffer)[:3]):
    s, a, r, s_next, done = exp
    print(f"  Exp {i+1}: s={s}, a={a}, r={r:.1f}, s'={s_next}, done={done}")
```

```
=====
EXPERIENCE REPLAY BUFFER
=====
```

Experiencias almacenadas: 5

¿Para qué sirve el Experience Replay?

1. Rompe la correlación temporal entre muestras consecutivas
2. Reutiliza experiencias pasadas (eficiencia de datos)
3. Estabiliza el entrenamiento de redes neuronales

Cada experiencia es una tupla (s, a, r, s', done):

Exp 1: s=[0 0], a=0, r=-0.1, s'=[1 2], done=False
Exp 2: s=[1 2], a=1, r=-0.1, s'=[2 4], done=False
Exp 3: s=[2 4], a=0, r=-0.1, s'=[3 6], done=False

3 2. Ejemplos Básicos (Nivel 2)

Ubicación: 02_fundamentos/ejemplos/

Los ejemplos son tutoriales completos que demuestran los algoritmos fundamentales.

3.1 2.1 Q-Learning Tabular: Taxi-v3

Archivo: 02_fundamentos/ejemplos/ejemplo_qlearning_taxi.py

3.1.1 Relación con la Teoría

Concepto	Aplicación en Taxi
Estados discretos	500 estados posibles ($5 \times 5 \times 5 \times 4$)
Acciones discretas	6 acciones (N, S, E, W, pickup, dropoff)
Tabla Q	Matriz 500×6
Recompensas	+20 entrega, -10 acción ilegal, -1 por paso
Episodios	Termina al entregar pasajero

3.1.2 Por qué funciona Q-Learning tabular aquí

- El espacio de estados es **pequeño y discreto** (500 estados)
- Podemos almacenar $Q(s,a)$ en una tabla real
- No necesitamos aproximación de funciones

```
[5]: # Visualizar el entorno Taxi-v3
import gymnasium as gym

print("="*60)
print("ENTORNO: Taxi-v3")
print("="*60)

env = gym.make("Taxi-v3")

print(f"""
Espacio de estados: {env.observation_space}
- 500 estados discretos
- Codifica: posición taxi (25) × posición pasajero (5) × destino (4)

```

```

Espacio de acciones: {env.action_space}
- 0: Sur
- 1: Norte
- 2: Este
- 3: Oeste
- 4: Recoger pasajero
- 5: Dejar pasajero

Recompensas:
- +20: Entregar pasajero correctamente
- -10: Recoger/dejar en lugar incorrecto
- -1: Cada paso (incentiva eficiencia)
"""

# Mostrar un estado inicial
obs, _ = env.reset(seed=42)
print(f"Estado inicial (codificado): {obs}")
print(f"\nVisualización ASCII:")
print(env.render())

env.close()

```

```

=====
ENTORNO: Taxi-v3
=====

```

```

Espacio de estados: Discrete(500)
- 500 estados discretos
- Codifica: posición taxi (25) × posición pasajero (5) × destino (4)

```

```

Espacio de acciones: Discrete(6)
- 0: Sur
- 1: Norte
- 2: Este
- 3: Oeste
- 4: Recoger pasajero
- 5: Dejar pasajero

```

```

Recompensas:
- +20: Entregar pasajero correctamente
- -10: Recoger/dejar en lugar incorrecto
- -1: Cada paso (incentiva eficiencia)

```

```

Estado inicial (codificado): 386

```

```

Visualización ASCII:
None

```



```
/home/jeironpro/Documentos/OPT-ML2/.venv/lib64/python3.11/site-
packages/gymnasium/envs/toy_text/taxi.py:443: UserWarning: WARN: You are
calling render method without specifying any render mode. You can specify the
render_mode at initialization, e.g. gym.make("Taxi-v3",
render_mode="rgb_array")
gym.logger.warn(
```

3.2 2.2 DQN: CartPole-v1

Archivo: 02_fundamentos/ejemplos/ejemplo_cartpole_dqn.py

3.2.1 Relación con la Teoría

Concepto	Aplicación en CartPole
Estados continuos	4 valores: posición, velocidad, ángulo, vel. angular
Por qué no tabular	Estados continuos → infinitos valores posibles
Red neuronal	Aproxima $Q(s,a)$ para todo s
Experience Replay	Buffer de 10,000 experiencias
Target Network	Actualizada cada 10 episodios

3.2.2 Arquitectura de la Red

Estado [4] → Dense(64) → ReLU → Dense(64) → ReLU → Q-valores [2]

(posición, velocidad,
ángulo, vel_angular) (izq, derecha)

```
[6]: # Visualizar el entorno CartPole
import gymnasium as gym
import numpy as np

print("="*60)
print("ENTORNO: CartPole-v1")
print("="*60)

env = gym.make("CartPole-v1")

print(f"""
Espacio de estados: {env.observation_space}
- Estado CONTINUO de 4 dimensiones:
  [0] Posición del carro      (-4.8 a 4.8)
  [1] Velocidad del carro     (-∞ a ∞)
  [2] Ángulo del poste        (-0.42 a 0.42 rad ±24°)
  [3] Velocidad angular       (-∞ a ∞)

Espacio de acciones: {env.action_space}
```

- 0: Empujar hacia la IZQUIERDA
- 1: Empujar hacia la DERECHA

Recompensa: +1 por cada paso que el poste sigue en pie

Termina cuando:

- El poste se inclina más de 12° (± 0.21 rad)
- El carro sale del área visible
- Se alcanzan 500 pasos (¡éxito!)

"""

Mostrar una observación

obs, _ = env.reset(seed=42)

print(f"Observación de ejemplo: {obs}")

print(f" Posición carro: {obs[0]:.4f}")

print(f" Velocidad carro: {obs[1]:.4f}")

print(f" Ángulo poste: {obs[2]:.4f} rad ({np.degrees(obs[2]):.2f}°)")

print(f" Velocidad angular: {obs[3]:.4f}")

env.close()

=====

ENTORNO: CartPole-v1

=====

Espacio de estados: Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)

- Estado CONTINUO de 4 dimensiones:

[0] Posición del carro (-4.8 a 4.8)

[1] Velocidad del carro ($-\infty$ a ∞)

[2] Ángulo del poste (-0.42 a 0.42 rad $\pm 24^\circ$)

[3] Velocidad angular ($-\infty$ a ∞)

Espacio de acciones: Discrete(2)

- 0: Empujar hacia la IZQUIERDA

- 1: Empujar hacia la DERECHA

Recompensa: +1 por cada paso que el poste sigue en pie

Termina cuando:

- El poste se inclina más de 12° (± 0.21 rad)
- El carro sale del área visible
- Se alcanzan 500 pasos (¡éxito!)

Observación de ejemplo: [0.0273956 -0.00611216 0.03585979 0.0197368]

Posición carro: 0.0274

Velocidad carro: -0.0061

Ángulo poste: 0.0359 rad (2.05°)

Velocidad angular: 0.0197

```
[7]: # Ver la arquitectura DQN del ejemplo
print("="*60)
print("ARQUITECTURA DQN PARA CARTPOLE")
print("="*60)

try:
    import torch
    from core.agentes import DQNetwork

    # Crear red para CartPole (4 inputs, 2 outputs)
    red = DQNetwork(input_size=4, n_acciones=2)

    print(f"\nArquitectura de la red:")
    print(red)

    # Contar parámetros
    n_params = sum(p.numel() for p in red.parameters())
    print(f"\nTotal de parámetros entrenables: {n_params:,}")

    # Hacer una predicción de ejemplo
    estado = torch.FloatTensor([[0.02, 0.01, 0.03, -0.02]])
    with torch.no_grad():
        q_valores = red(estado)

    print(f"\nEjemplo de predicción:")
    print(f" Estado: {estado.numpy()[0]}")
    print(f" Q(s, izq): {q_valores[0, 0]:.4f}")
    print(f" Q(s, der): {q_valores[0, 1]:.4f}")
    print(f" Mejor acción: {'DERECHA' if q_valores[0, 1] > q_valores[0, 0]
↪else 'IZQUIERDA'}")

except ImportError as e:
    print(f"PyTorch no disponible: {e}")
```

```
=====
ARQUITECTURA DQN PARA CARTPOLE
=====
```

Arquitectura de la red:

```
DQNetwork(
  (network): Sequential(
    (0): Linear(in_features=4, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=2, bias=True)
```

```
)  
)
```

Total de parámetros entrenables: 17,410

Ejemplo de predicción:

Estado: [0.02 0.01 0.03 -0.02]

Q(s, izq): -0.0144

Q(s, der): -0.0696

Mejor acción: IZQUIERDA

4 3. Proyectos DQN (Nivel 3)

Ubicación: 03_proyectos_dqn/

Proyectos que aplican DQN en escenarios visuales con Pygame.

4.1 Mapa de Proyectos

Proyecto	Algoritmo	Librería	Dificultad	Tipo de Observación
Nibbler	DQN	PyTorch		Grid/Estado discreto
Racing	DQN	PyTorch		Sensores continuos
Flappy Bird	DQN/PPO	SB3		Vector o píxeles

4.2 3.1 Nibbler (Snake) - DQN desde Cero

Directorio: 03_proyectos_dqn/nibbler/

4.2.1 Descripción

El clásico juego de la serpiente (Snake) implementado con Pygame, donde un agente DQN aprende a comer manzanas y evitar chocar.

4.2.2 Relación con la Teoría

Concepto Teórico	Implementación en Nibbler
Estado	Vector con: dirección, posición relativa de comida, peligros cercanos
Acciones	3 acciones: recto, girar izquierda, girar derecha
Recompensa	+10 comer, -10 morir, +1 acercarse a comida
Red neuronal	MLP: estado $\rightarrow$ capas ocultas $\rightarrow$ 3 Q-valores
Experience Replay	Sí, buffer de experiencias
Target Network	Sí, actualizada cada N pasos

4.2.3 Diseño del Estado (Ingeniería de Features)

```
Estado = [  
    peligro_adelante, # ¿Hay pared/cuerpo adelante?  
    peligro_izquierda, # ¿Hay peligro a la izquierda?  
    peligro_derecha, # ¿Hay peligro a la derecha?  
    direccion_arriba, # ¿Voy hacia arriba?  
    direccion_abajo, # ¿Voy hacia abajo?  
    direccion_izq, # ¿Voy hacia la izquierda?  
    direccion_der, # ¿Voy hacia la derecha?  
    comida_arriba, # ¿Comida está arriba?  
    comida_abajo, # ¿Comida está abajo?  
    comida_izq, # ¿Comida está a la izquierda?  
    comida_der # ¿Comida está a la derecha?  
]
```

Nota: Este es un buen ejemplo de cómo el **diseño del estado** afecta el aprendizaje. Un estado bien diseñado facilita que el agente aprenda.

```
[8]: # Verificar archivos del proyecto Nibbler  
from pathlib import Path  
  
nibbler_dir = RL_DIR / "03_proyectos_dqn" / "nibbler"  
  
print("="*60)  
print("PROYECTO: Nibbler (Snake)")  
print("="*60)  
  
if nibbler_dir.exists():  
    print(f"\nArchivos en {nibbler_dir.relative_to(RL_DIR)}:")  
    for f in sorted(nibbler_dir.iterdir()):  
        if f.is_file():  
            size = f.stat().st_size / 1024  
            print(f"    {f.name} ({size:.1f} KB)")  
else:  
    print(f"Directorio no encontrado: {nibbler_dir}")  
  
# Verificar modelo guardado  
modelo_nibbler = RL_DIR / "modelos" / "nibbler_best.pth"  
if modelo_nibbler.exists():  
    size = modelo_nibbler.stat().st_size / 1024  
    print(f"\n Modelo entrenado disponible: modelos/nibbler_best.pth ({size:.  
↪1f} KB)")  
else:  
    print(f"\n Modelo no encontrado")
```

```
=====
```

```
PROYECTO: Nibbler (Snake)
```

```
=====
```

Archivos en 03_proyectos_dqn/nibbler:

nibbler_best.pth (17290.5 KB)
nibbler_game.py (25.5 KB)
nibbler_rl.ipynb (286.0 KB)
nibbler_training.png (227.9 KB)

Modelo entrenado disponible: modelos/nibbler_best.pth (17290.6 KB)

4.3 3.2 Racing (Carreras Multi-Agente)

Directorio: 03_proyectos_dqn/racing/

4.3.1 Descripción

Juego de carreras con múltiples coches (4-8) que aprenden simultáneamente a navegar un circuito.

4.3.2 Relación con la Teoría

Concepto	Aplicación
Multi-agente	Cada coche es un agente independiente
Estado	Sensores de distancia (raycast) + velocidad
Acciones	Acelerar, frenar, girar
Reward shaping	+1 avanzar, -100 chocar, bonus por velocidad

```
[9]: # Verificar modelos de racing
from pathlib import Path

print("="*60)
print("PROYECTO: Racing (Multi-Agente)")
print("="*60)

modelos_dir = RL_DIR / "modelos"
if modelos_dir.exists():
    car_models = list(modelos_dir.glob("car_agent_*.pth"))
    print(f"\nModelos de coches entrenados: {len(car_models)}")
    for m in sorted(car_models):
        size = m.stat().st_size / 1024
        print(f"    {m.name} ({size:.1f} KB)")

print(f"""
Características del proyecto:
- 4-8 coches compitiendo simultáneamente
- Cada coche tiene sensores de distancia (raycast)
- Aprenden a mantener la trayectoria y evitar colisiones
- Visualización con Pygame
""")
```

```
=====
PROYECTO: Racing (Multi-Agente)
=====
```

Modelos de coches entrenados: 4

```
car_agent_0.pth (75.6 KB)
car_agent_1.pth (75.6 KB)
car_agent_2.pth (75.6 KB)
car_agent_3.pth (75.6 KB)
```

Características del proyecto:

- 4-8 coches compitiendo simultáneamente
- Cada coche tiene sensores de distancia (raycast)
- Aprenden a mantener la trayectoria y evitar colisiones
- Visualización con Pygame

5 4. Proyectos Avanzados (Nivel 4)

Ubicación: 04_proyectos_avanzados/

Proyectos que usan algoritmos avanzados (PPO, SAC, TD3) con Stable-Baselines3.

5.1 Mapa de Proyectos Avanzados

Proyecto	Algoritmo	Dificultad	Tipo de Observación
LunarLander	PPO/DQN/A2C		Vector 8D
Highway	DQN/PPO		Cinemático 5×5
MiniGrid	PPO		Imagen 7×7×3
PyBullet	PPO/SAC/TD3		Sensores robóticos

5.2 4.1 LunarLander - PPO con Stable-Baselines3

Directorio: 04_proyectos_avanzados/lunarlander/

5.2.1 Descripción

Aterrizar una nave espacial en la superficie lunar usando propulsores.

5.2.2 PPO vs DQN

Aspecto	DQN	PPO
Tipo	Value-based	Policy-based
Acciones	Solo discretas	Discretas y continuas
Estabilidad	Menos estable	Muy estable

Aspecto	DQN	PPO
Eficiencia de datos	Mejor (replay)	Peor (on-policy)

PPO (Proximal Policy Optimization) es uno de los algoritmos más usados en la práctica por su robustez.

```
[10]: # Ejemplo de código de LunarLander con SB3
print("="*60)
print("LUNARLANDER CON STABLE-BASELINES3")
print("="*60)

codigo_lunar = '''
# Código simplificado de 04_proyectos_avanzados/lunarlander/lunarlander_sb3.py

from stable_baselines3 import PPO, DQN, A2C
import gymnasium as gym

# 1. Crear entorno
env = gym.make("LunarLander-v3")

# 2. Crear agente (;solo 1 línea!)
modelo = PPO(
    "MlpPolicy",          # Red neuronal MLP
    env,
    learning_rate=0.0003,
    gamma=0.99,
    verbose=1
)

# 3. Entrenar
modelo.learn(total_timesteps=100_000)

# 4. Guardar
modelo.save("lunarlander_ppo")

# 5. Demo
env = gym.make("LunarLander-v3", render_mode="human")
obs, _ = env.reset()
while True:
    action, _ = modelo.predict(obs, deterministic=True)
    obs, reward, done, _, _ = env.step(action)
    if done:
        break
'''
print(codigo_lunar)
```



```
print("\n¡Stable-Baselines3 abstrae toda la complejidad!")
print(" - Experience Replay: Automático")
print(" - Target Network: Automático")
print(" - Normalización: Automático")
print(" - Logging: Automático")
```

=====

LUNARLANDER CON STABLE-BASELINES3

=====

Código simplificado de 04_proyectos_avanzados/lunarlander/lunarlander_sb3.py

```
from stable_baselines3 import PPO, DQN, A2C
import gymnasium as gym
```

1. Crear entorno

```
env = gym.make("LunarLander-v3")
```

2. Crear agente (¡solo 1 línea!)

```
modelo = PPO(
    "MlpPolicy",          # Red neuronal MLP
    env,
    learning_rate=0.0003,
    gamma=0.99,
    verbose=1
)
```

3. Entrenar

```
modelo.learn(total_timesteps=100_000)
```

4. Guardar

```
modelo.save("lunarlander_ppo")
```

5. Demo

```
env = gym.make("LunarLander-v3", render_mode="human")
obs, _ = env.reset()
while True:
    action, _ = modelo.predict(obs, deterministic=True)
    obs, reward, done, _, _ = env.step(action)
    if done:
        break
```

¡Stable-Baselines3 abstrae toda la complejidad!

- Experience Replay: Automático
- Target Network: Automático
- Normalización: Automático
- Logging: Automático

5.3 4.2 Otros Proyectos Avanzados

5.3.1 Highway (Conducción Autónoma)

- **Directorio:** 04_proyectos_avanzados/highway/
- **Observación cinemática:** Matriz 5×5 con vehículos cercanos
- **Escenarios:** autopista, merge, roundabout, parking

5.3.2 MiniGrid (Laberintos)

- **Directorio:** 04_proyectos_avanzados/minigrid/
- **Observación parcial:** El agente solo ve 7×7 celdas frente a él
- **Wrappers:** Transforman la observación (flatten, RGB)

5.3.3 PyBullet (Robótica 3D)

- **Directorio:** 04_proyectos_avanzados/pybullet/
 - **Acciones continuas:** Torques en cada articulación
 - **Algoritmos:** SAC, TD3 (para acciones continuas)
-

6 5. Modelos Guardados

Ubicación: modelos/

El directorio contiene pesos de redes neuronales ya entrenadas.

```
[11]: # Listar todos los modelos guardados
from pathlib import Path

print("="*60)
print("MODELOS GUARDADOS")
print("="*60)

modelos_dir = RL_DIR / "modelos"

if modelos_dir.exists():
    modelos = list(modelos_dir.glob("*.pth")) + list(modelos_dir.glob("*.zip"))

    print(f"\nModelos encontrados: {len(modelos)}\n")

    for m in sorted(modelos):
        size = m.stat().st_size / 1024

        # Determinar tipo y proyecto
        if "cartpole" in m.name.lower():
            proyecto = "CartPole (DQN) - Nivel 2"
            icono = " "
        elif "nibbler" in m.name.lower():
```

```

        proyecto = "Nibbler/Snake (DQN) - Nivel 3"
        icono = " "
    elif "car_agent" in m.name.lower():
        proyecto = "Racing (Multi-agente) - Nivel 3"
        icono = " "
    elif "lunar" in m.name.lower():
        proyecto = "LunarLander (PPO/DQN) - Nivel 4"
        icono = " "
    else:
        proyecto = "Desconocido"
        icono = " "

    print(f"{icono} {m.name}")
    print(f"    Proyecto: {proyecto}")
    print(f"    Tamaño: {size:.1f} KB")
    print()
else:
    print("Directorio de modelos no encontrado")

```

=====

MODELOS GUARDADOS

=====

Modelos encontrados: 6

```

car_agent_0.pth
  Proyecto: Racing (Multi-agente) - Nivel 3
  Tamaño: 75.6 KB

car_agent_1.pth
  Proyecto: Racing (Multi-agente) - Nivel 3
  Tamaño: 75.6 KB

car_agent_2.pth
  Proyecto: Racing (Multi-agente) - Nivel 3
  Tamaño: 75.6 KB

car_agent_3.pth
  Proyecto: Racing (Multi-agente) - Nivel 3
  Tamaño: 75.6 KB

cartpole_dqn.pth
  Proyecto: CartPole (DQN) - Nivel 2
  Tamaño: 281.6 KB

nibbler_best.pth
  Proyecto: Nibbler/Snake (DQN) - Nivel 3
  Tamaño: 17290.6 KB

```

7 6. Resumen: De la Teoría a la Práctica

7.1 Flujo de Aprendizaje por Niveles

NIVEL 1: TEORÍA

01_teoría/RL_01_fundamentos.ipynb

Conceptos: Agente, Estado, Acción, Recompensa

MDP y Ecuación de Bellman

Q-Learning y DQN

NIVEL 2: FUNDAMENTOS

02_fundamentos/core/agentes.py

Ver cómo se implementa Q-Learning

Entender ReplayBuffer

Estudiar arquitectura DQN

02_fundamentos/ejemplos/

ejemplo_qlearning_taxi.py → Q-Learning tabular

ejemplo_cartpole_dqn.py → DQN básico

NIVEL 3: PROYECTOS DQN

03_proyectos_dqn/

nibbler/ → DQN desde cero con Pygame

flappybird/ → DQN/PPO

racing/ → Multi-agente

NIVEL 4: PROYECTOS AVANZADOS

04_proyectos_avanzados/

lunarlander/ → Stable-Baselines3 (PPO, A2C)

highway/ → Conducción autónoma

minigrid/ → Observación parcial

pybullet/ → Robótica 3D (SAC, TD3)

7.2 Tabla de Correspondencia Completa

Concepto Teórico	Dónde se Aplica	Archivo de Referencia
Ciclo Agente-Entorno	Todos los proyectos	02_fundamentos/core/agentes.py
Política -greedy	Q-Learning, DQN	02_fundamentos/core/agentes.py
Ecuación de Bellman	Q-Learning	02_fundamentos/core/agentes.py
Tabla Q	Taxi	02_fundamentos/ejemplos/ejemplo_qlearning_t
Estados continuos	CartPole, LunarLander	02_fundamentos/ejemplos/ejemplo_cartpole_do
Red Neuronal para Q	DQN	02_fundamentos/core/agentes.py:DQNNetwork
Experience Replay	DQN	02_fundamentos/core/agentes.py:ReplayBuffer
Target Network	DQN	02_fundamentos/core/agentes.py:DQNAgent
PPO	LunarLander, Highway	04_proyectos_avanzados/lunarlander/

Concepto Teórico	Dónde se Aplica	Archivo de Referencia
SAC/TD3	PyBullet (robótica)	04_proyectos_avanzados/pybullet/
Multi-agente	Racing	03_proyectos_dqn/racing/
Wrappers	MiniGrid	04_proyectos_avanzados/minigrid/

7.3 Comandos Rápidos para Ejecutar Proyectos

Desde el directorio 'Reinforcement Learning/'

Nivel 2: Ejemplos básicos

python 02_fundamentos/ejemplos/ejemplo_qlearning_taxi.py

python 02_fundamentos/ejemplos/ejemplo_cartpole_dqn.py

Nivel 3: Proyectos DQN

python 03_proyectos_dqn/nibbler/nibbler_game.py

Jugar

python 03_proyectos_dqn/nibbler/nibbler_game.py --train

Entrenar

python 03_proyectos_dqn/racing/racing_game.py

Carreras

python 03_proyectos_dqn/flappybird/flappybird_dqn.py

Flappy Bird

Nivel 4: Proyectos Avanzados

python 04_proyectos_avanzados/lunarlander/lunarlander_sb3.py

Entrenar

python 04_proyectos_avanzados/lunarlander/lunarlander_sb3.py --demo

Demo

python 04_proyectos_avanzados/highway/highway_conduccion.py

python 04_proyectos_avanzados/minigrid/minigrid_navegacion.py

python 04_proyectos_avanzados/pybullet/pybullet_robotica.py

8 Conclusión

Este directorio de Reinforcement Learning está organizado en **4 niveles de complejidad**:

1. **Nivel 1 (Teoría)** → 01_teoría/ - Notebooks con fundamentos teóricos
2. **Nivel 2 (Fundamentos)** → 02_fundamentos/ - Implementaciones base y ejemplos
3. **Nivel 3 (DQN)** → 03_proyectos_dqn/ - Proyectos visuales con DQN
4. **Nivel 4 (Avanzado)** → 04_proyectos_avanzados/ - PPO, SAC, TD3

Cada proyecto demuestra conceptos específicos de RL en acción, permitiendo ver cómo la matemática se traduce en código funcional.

¡Experimenta, modifica, y crea tus propios agentes!